Sorting algorithms

Insertion Sort

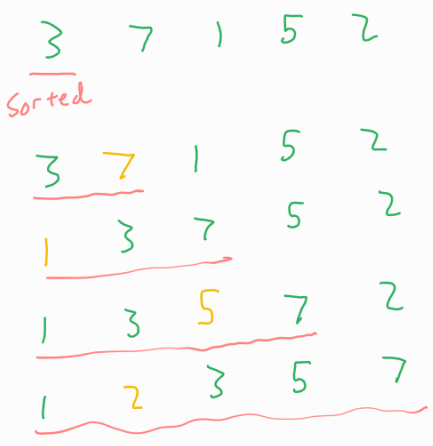
these preceding elements will be sorted from previous iterations

- Starting with the second element, compare each element with the preceding elements until a larger element is found, and insert it in the correct place in the sorted sublist of preceding elements.
- After j^{th} iteration, first $j+1$ elements of the original list are in order.
- running time between $n-1$ (sorted case) and $\frac{n(n-1)}{2}$ (reverse sorted case)
  => $O(n^2)$ worst case

InsertionSort $(a_1, \ldots, a_n \in \mathbb{R}, n \geq 2)$

```
1: for j = 2 to n do
2:     x = a_j
3:     i = j − 1
4:     while i > 0 and a_i > x do
5:         a_{i+1} = a_i
6:         i = i − 1
7:     end while
8:     a_{i+1} = x
9: end for
```

Example



Selection Sort

- Find the smallest element and swap with the first element.
- i^{th} iteration: Find the smallest unsorted element and swap with the i^{th} element
- After i iterations, the positions $1, \ldots, i$ contain the $1^{st} \ldots i^{th}$ smallest overall elements of the list in order.
- Time complexity: $\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} (n-i)$

$$= \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i$$

$$= (n-1)n - \frac{n(n-1)}{2} = \frac{n(n-1)}{2}$$

$$\Rightarrow O(n^2)$$

SelectionSort $(a_1, \ldots, a_n \in \mathbb{R}, n \geq 2)$

```
1: for i = 1 to n − 1 do
2:     elem = a_i
3:     pos = i
4:     for j = i + 1 to n do
5:         if a_j < elem then
6:             elem = a_j   -> value of smallest remaining element
7:             pos = j      -> position of smallest rem. elem.
8:         end if
9:     end for
10:    swap a_i and a_pos
11: end for
```

Example

1 2 3 5 7

1 2 3 5 7

1 2 3 5 7

Bubble Sort

- Sort using multiple passes, with each pass comparing elements pairwise and swapping them if they are in the wrong order
- After the i^{th} iteration of the outer loop, the positions $i, \dots, n$ contain the $(n-i)+1$ largest elements in order.
- There exists a variation that terminates if no swaps were performed in a pass -> this signifies the list is sorted.
- Time complexity: $(n-1)+(n-2)+\dots+2+1 = \frac{(n-1)n}{2}$

$\Rightarrow O(n^2)$

Example

3 7 1 5 2

3 7 1 5 2 → 1 3 5 2 7 → 1 3 2 5 7 → 1 2 3 5 7

3 1 7 5 2 → 1 3 5 2 7 → 1 2 3 5 7

3 1 5 7 2 → 1 3 2 5 7

3 1 5 2 7 → 1 3 2 5 7 sorted

BubbleSort $(a_1, \dots, a_n \in \mathbb{R}, n \geq 2)$

```
1: for i = n downto 2 do
2:     for j = 2 to i do
3:         if a_{j-1} > a_j then
4:             swap a_{j-1} and a_j
5:         end if
6:     end for
7: end for
```

Merge Sort

- If input list has length = 1, just return the input.
- else, split into two equal length sublists and sort them recursively. Then, merge the resulting sorted sublists into a single sorted list. This recursive process sorts the original list.
- To merge two sorted lists, repeatedly add the smallest of the leftmost elements of the two lists to the result until all elements from both lists are appended to the result.
- Time complexity:

Recurrence: $T(n) = \underbrace{2T(n/2)}_{\text{recursion}} + \underbrace{cn}_{\text{merging}}$

Master Theorem with $a=2$, $b=2$, $f(n)=cn$:

$\log_b a = \log_2 2 = 1$

$\therefore n^{\log_b a} = n$

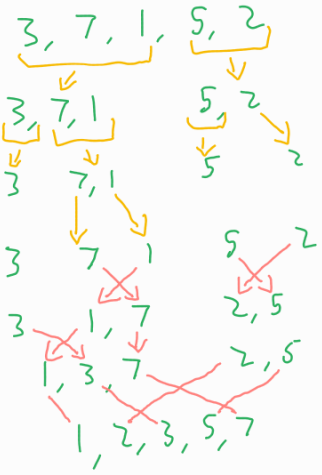
$f(n) = cn = \Theta(n \log^k n)$ for $k=0$

$\therefore T(n) = \Theta(n \log n)$ by case 2 of the Master Theorem.

$\Rightarrow$ Merge Sort is $\Theta(n \log n)$

- In some specific inputs, Insertion Sort can outperform Merge Sort, as MS is always $\Theta(n \log n)$, whereas the best case for IS is $O(n)$.

Example



```
list MergeSort (list m)
1:  if length(m) ≤ 1 then
2:      return m
3:  end if
4:  int middle = length(m) / 2
5:  list left, right, leftsorted, rightsorted
6:  left = m[1..middle]
7:  right = m[middle+1..length(m)]
8:  leftsorted = MergeSort(left)
9:  rightsorted = MergeSort(right)
10: return Merge(leftsorted, rightsorted)
```

```
list Merge (list left, list right)
1:  list result
2:  while length(left) > 0 or length(right) > 0 do
3:      if length(left) > 0 and length(right) > 0 then
4:          if first(left) ≤ first(right) then
5:              append first(left) to result
6:              left = rest(left)
7:          else
8:              append first(right) to result
9:              right = rest(right)
10:         end if
11:     else if length(left) > 0 then
12:         append left to result
13:         left = empty list
14:     else      // length(right) > 0
15:         append right to result
16:         right = empty list
17:     end if
18: end while
19: return result
```

Quick Sort

- At the beginning of each recursive call, pick a pivot element and divide the input list into two partitions: one containing elements less than the pivot and the other containing elements greater than the pivot.

Note: Quick Sort sorts 'in place' — no new arrays are created at any point, it just reorders elements in the original input array.

- Quick Sort recursively partitions the list according to the partition function until the list is sorted. The partition function itself does not sort, but recursive application of it does sort the list.

- Time Complexity: Worst Case: Θn^2

In worst case, partition on list of size n always produces a partition of size 0 and the other of size $n-1$ (the pivot is not included in either partition)

Recurrence: $T(n) = \begin{cases} d & \text{base case} \\ T(q) + T(n-q-1) + an & 0 \le q \le n-1 \end{cases}$

Where q is the size of one partition.

Setting $q = 0$ gives $T(n) = T(n-1) + an$

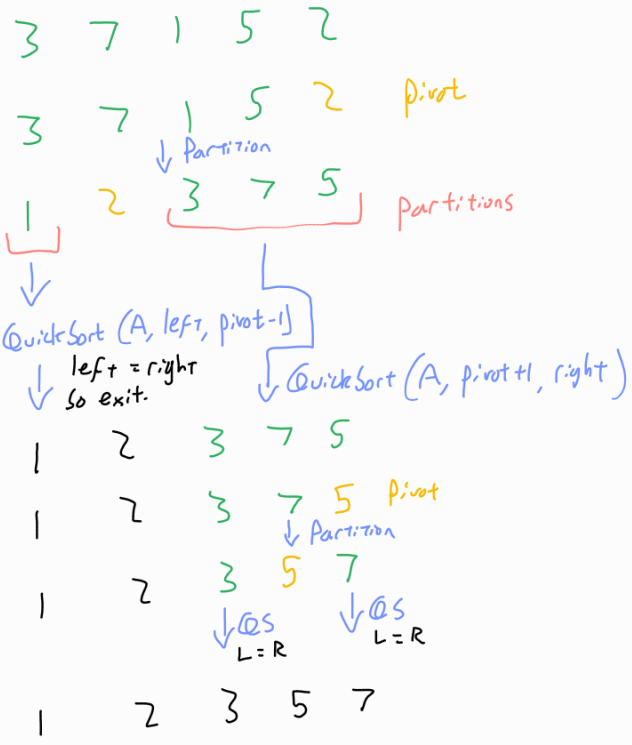
Guessing $T(n) \le \alpha n^2$ for some $\alpha > 0$ gives:

$$T(n) \le \alpha(n-1)^2 + an$$
$$= \alpha n^2 - 2\alpha n + \alpha + an$$
$$= \alpha n^2 - (2\alpha n - \alpha - an)$$
$$\le \alpha n^2 \text{ for } \alpha \gg \frac{an}{2n-1} = \Theta(1)$$

$\therefore$ worst case running time $O(n^2)$

Average case: $\Theta(n \log n)$

Example (choosing rightmost element as pivot)



QuickSort (int A[1...n], int left, int right)

```
1:  if (left < right) then
2:      // rearrange/partition in place
3:      // return value "pivot" is index of pivot element
4:      // in A[] after partitioning
5:      pivot = Partition (A, left, right)
6:      // Now:
7:      // everything in A[left...pivot-1] is smaller than pivot
8:      // everything in A[pivot+1...right] is bigger than pivot
9:      QuickSort (A, left, pivot-1)
10:     QuickSort (A, pivot+1, right)
11: end if
```

int **Partition** (A[1...n], int left, int right)

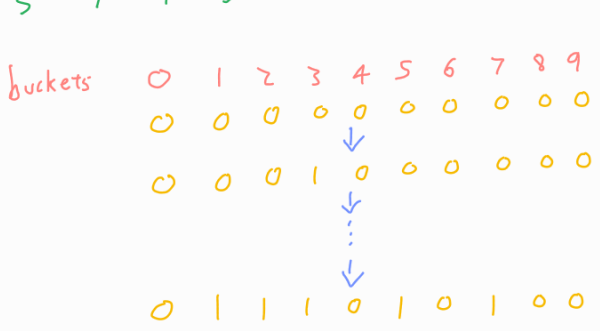
```
1:  int x = A[right]
2:  int i = left-1
3:  for j=left to right-1 do
4:      if A[j] < x then
5:          i = i+1
6:          swap A[i] and A[j]
7:      end if
8:  end for
9:  swap A[i+1] and A[right]
10: return i+1    // pivot position after partitioning
```

Since @S sorts the array in place, left and right are pointers to the leftmost and rightmost elements of the subarray we are considering in a given recursion.

Bucket Sort

- Suppose our input values are only within a known range $0 \ldots k-1$.
- Initialise an array of k "buckets", all initialised to 0.
- The indices of the buckets represent the values, the internal value inside each bucket represents the frequency of that value within the input list.
- Iterate through the input list, incrementing each element's corresponding bucket accordingly.
- Finally iterate through the buckets in order and place the corresponding elements back into the input list → it is now sorted.

Example ($k=9$)



- Time Complexity:
  - $O(k)$ for bucket initialisation
  - $O(n)$ for filling the buckets
  - $O(n)$ for constructing the final sorted list.

∴ overall $O(n+k)$ in worst case

If k small compared to n: $O(n)$

If k significantly larger than n, k dominates so it becomes $O(k)$ → if $k > n^2$ then BS is $\omega(n^2)$ → garbage.

Note: If not sorting numbers we can sort key:value pairs by having the buckets being queues and enqueueing values in the bucket corresponding to the key.

```
Algorithm BucketSort( S )
( values in S are between 0 and k-1 )
for j = 0 to k-1 do      // initialize k buckets
    b[j] = 0
end for
for i = 0 to n-1 do          // place elements in their
    b[S[i]] = b[S[i]] + 1    // appropriate buckets
end for
i = 0
for j = 0 to k-1 do      // place elements in buckets
    for r = 1 to b[j] do     // back in S
      S[i] = j
      i = i + 1
    end for
end for
```

Radix Sort

- As Bucket Sort but have as many buckets as you have different digits, and repeatedly bucket sort by digit, starting with the rightmost digit and working left.
- Note: buckets are queues here -> 2 numbers with the same digit should be in the same order after a pass as they were before that pass.

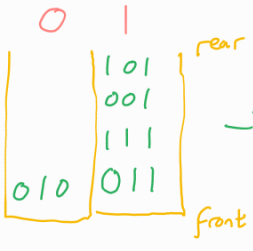
- Time Complexity: $O(n \log k)$ where k is the size of the range of possible values.
- Compared to Bucket Sort for $\{0, \ldots, k-1\}$:
  - RS: $O(n \log k)$
  - BS: $O(n+k)$
  - If k constant, both are $O(n)$
  - If k is $O(n)$ then RS is $O(n^2)$, BS is $O(n)$
  - If k large compared to n, RS is faster.

Example (binary)

011 111 001 101 010

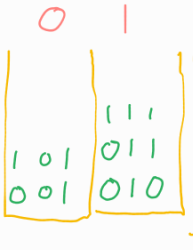
buckets

rightmost digit:



-> 010 011 111 001 101

middle digit:



-> 001 101 010 011 111

first digit

-> 001 010 011 101 111

Heap Sort

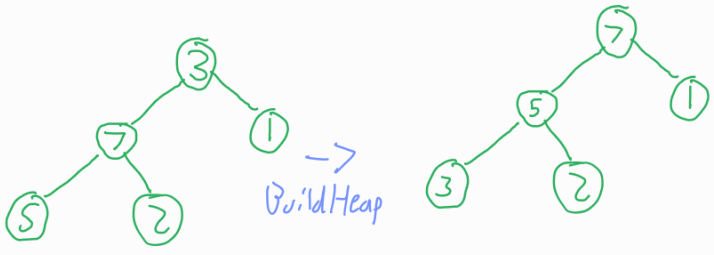
- HeapSort uses the array implementation of the Heap data structure to sort the elements of the array in-place.
- Build a max heap, then repeatedly:
  - Swap the root and last element
  - Reduce the heap size by 1 — this "removes" the last element from the heap while keeping it in the array. Since it is a max heap, this puts the old root, i.e. largest, element at the end of the array, where it will stay as it is no longer in the heap.
  - Heapify to maintain the heap property
- Time complexity $O(n \log n)$

HeapSort(A)

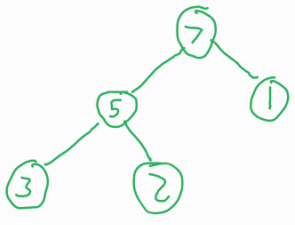
```
BuildHeap(A, Length(A))
for i = Length(A) downto 2 do
  swap A[i] and A[1]
  HeapSize(A) = HeapSize(A)-1
  Heapify(A, 1, HeapSize(A))
endfor
```

Example

3 7 1 5 2



Array: 7 5 1 3 2



Array: 7 5 1 3 2

i = 5

Array: 2 5 1 3 7

5 3 1 2 7

Sorted

i = 4

Array: 2 3 1 5 7

3 2 1 5 7

i = 3

Array: 1 2 3 5 7

2 1 3 5 7

i = 2

Array: 1 2 3 5 7

1 2 3 5 7 end